VAPT REPORT

Project Title:

Security Testing of a Simulated Web Application using DVWA

Target Application:

Damn Vulnerable Web Application

Tester: Kishori Bhaskar Kor

Environment: Local Lab (XAMPP, Browser)

INTRODUCTION

This project demonstrates security testing on DVWA (Damn Vulnerable Web Application), a deliberately vulnerable web application designed for learning cybersecurity. The focus is on identifying and exploiting common web vulnerabilities, such as file upload flaws, in a safe and controlled environment. By working on this project, we gain practical experience in detecting security weaknesses, testing exploits, and understanding how insecure applications can be compromised, helping to build awareness of web application security.

Disclaimer

This testing was performed in a controlled lab environment using DVWA for educational purposes only. No real systems were targeted.

OBJECTIVE

- Understand common web application vulnerabilities
- Perform penetration testing in a safe lab environment
- Demonstrate exploitation techniques
- Learn mitigation methods for improving security

TOOLS USED

1. Damn Vulnerable Web Application (DVWA)

A vulnerable web application used for practicing security testing. It provides a safe environment to learn common attacks like SQL injection, XSS, and file upload vulnerabilities.

2. XAMPP

A local server environment that includes Apache and MySQL, used to host DVWA on your machine.

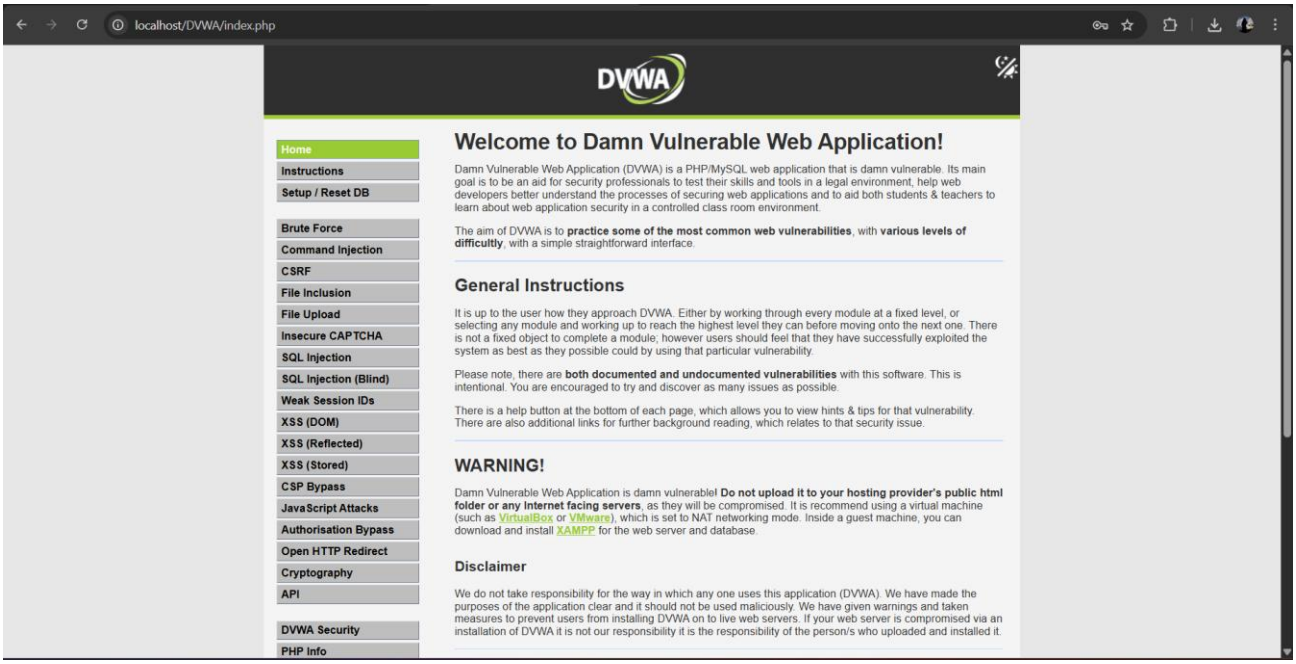
3. Browser Developer Tools

Tools available in modern browsers to inspect, debug, and manipulate web pages for testing purposes.

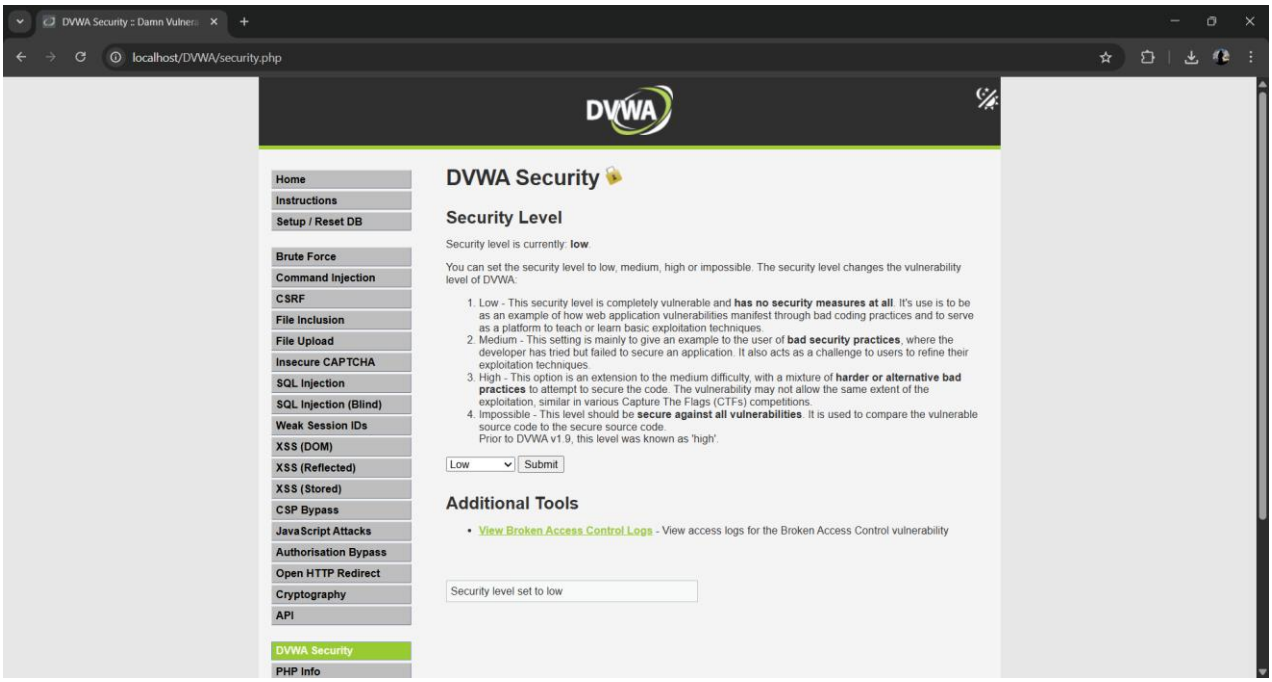
LAB SETUP

Steps to set up DVWA locally:

1. Installed XAMPP on your computer.
2. Started **Apache** and **MySQL** services from the XAMPP control panel.
3. Placed the DVWA folder inside the htdocs directory of XAMPP.
4. Opened DVWA in the browser using http://localhost/dvwa.
5. Configured the database using the DVWA setup page.
6. Logged in as admin using default credentials.



7. Security level set to Low



8. Lab Set up Completed

VULNERABILITY TESTING

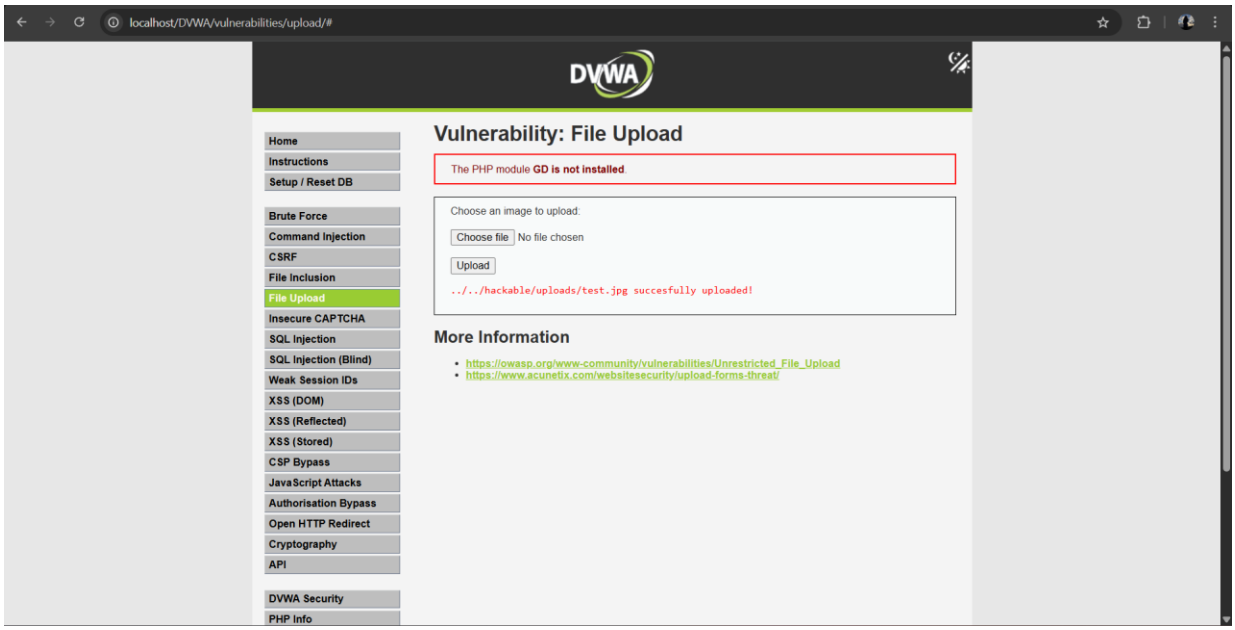
1. File Upload Vulnerability

Description: Allows attackers to upload malicious files that can be executed on the server.

Steps:

- Navigated to DVWA File Upload page.
- Created a PHP shell file (e.g., shell.php).
- Uploaded the shell.
- Accessed the uploaded file from the uploads folder.

Result: The server executed the file, confirming unrestricted file upload vulnerability.



Risk Level: High

Mitigation:

- Allow only specific file types (e.g., .jpg, .png, .pdf)
- Validate file extensions and MIME types
- Rename uploaded files automatically
- Store uploads outside the web root directory
- Disable execution permissions for upload folders

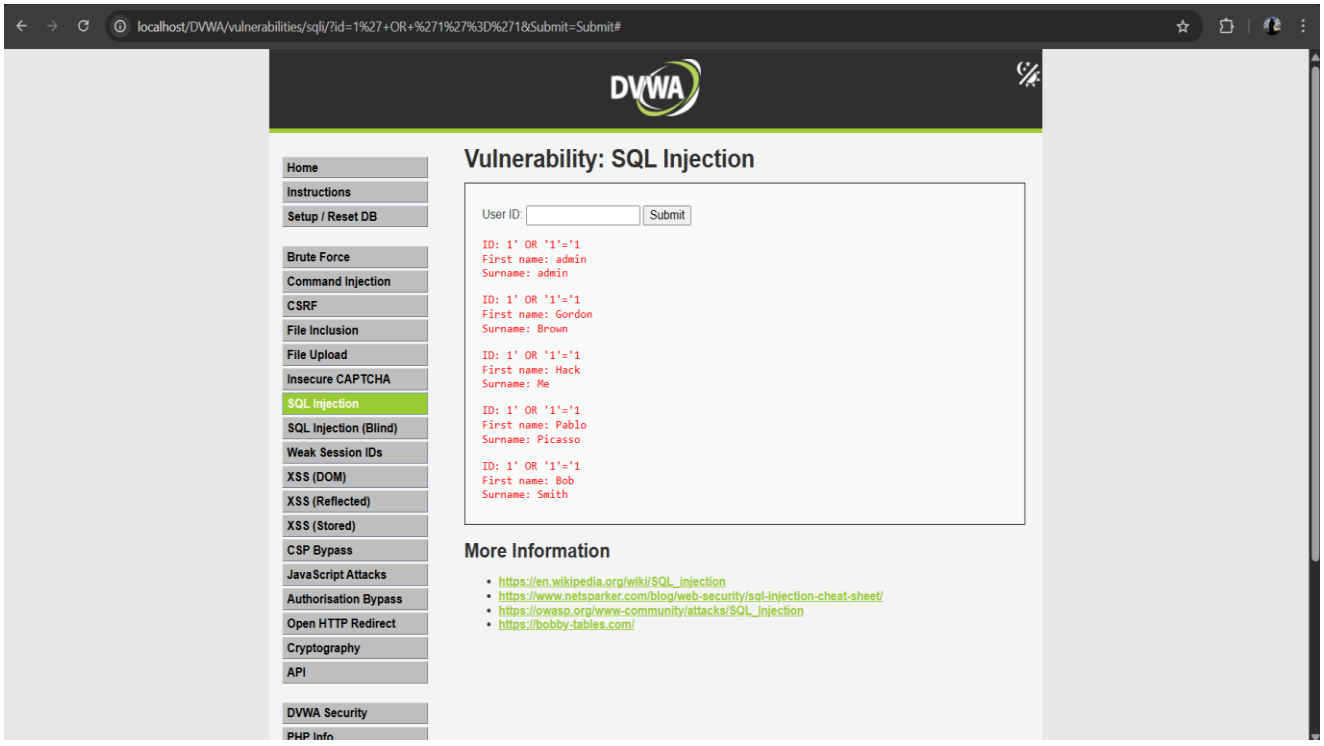
2. SQL Injection (SQLi)

Description: Attackers can manipulate database queries to access or modify data.

Steps:

- Navigated to the DVWA SQL Injection page.
- Entered special SQL characters or statements (e.g., ' OR '1'='1).
- Observed if the database returned unauthorized data.

Result: Successfully retrieved database records, showing the application is vulnerable.



Risk Level: High

Mitigation:

- Use **prepared statements / parameterized queries**
- Implement **input validation and sanitization**
- Use **ORM frameworks**
- Apply **least privilege access for database users**
- Use **Web Application Firewalls (WAF)**

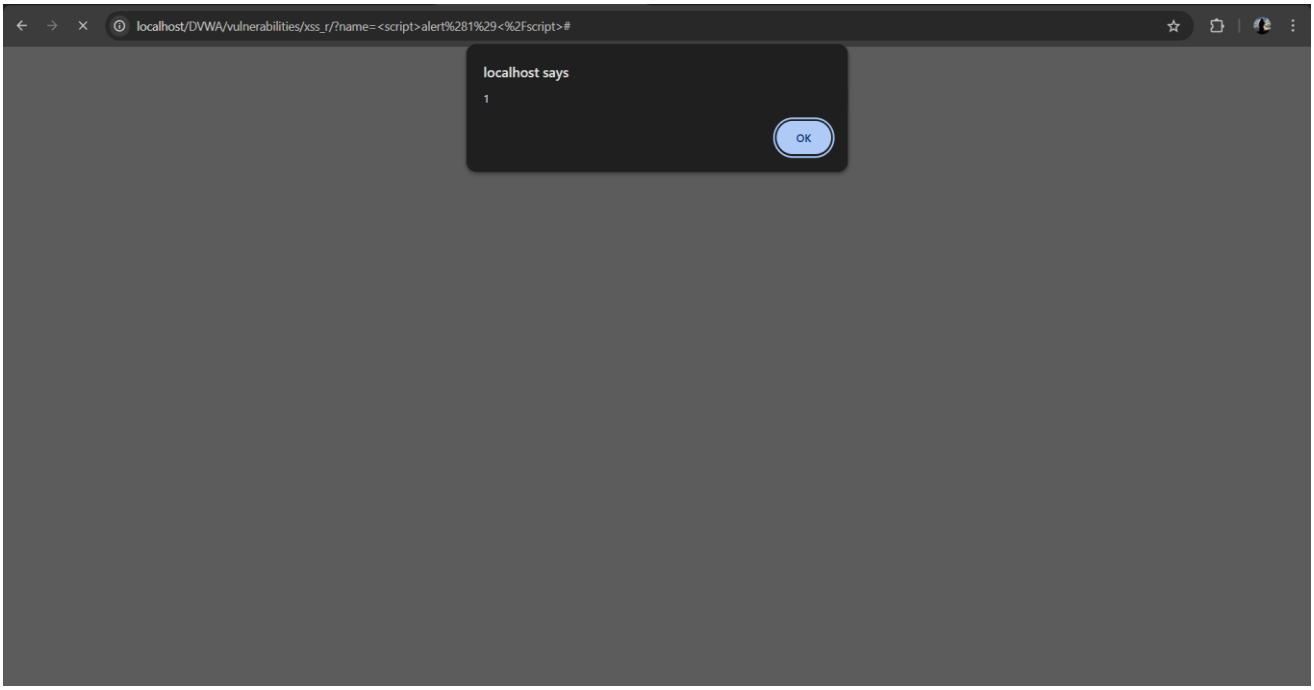
3. Cross-Site Scripting (XSS)

Description: Allows attackers to inject malicious scripts into web pages viewed by others.

Steps:

- Opened DVWA XSS (Reflected) page.
- Entered a JavaScript payload in the input field (e.g., <script>alert(‘1’)</script>).
- Submitted the input.

Result: The script executed in the browser, confirming XSS vulnerability.



Risk Level: Medium

Mitigation:

- Validate and sanitize user inputs
- Encode output before displaying it on web pages
- Use **Content Security Policy (CSP)**
- Disable inline JavaScript where possible

4. Command Injection

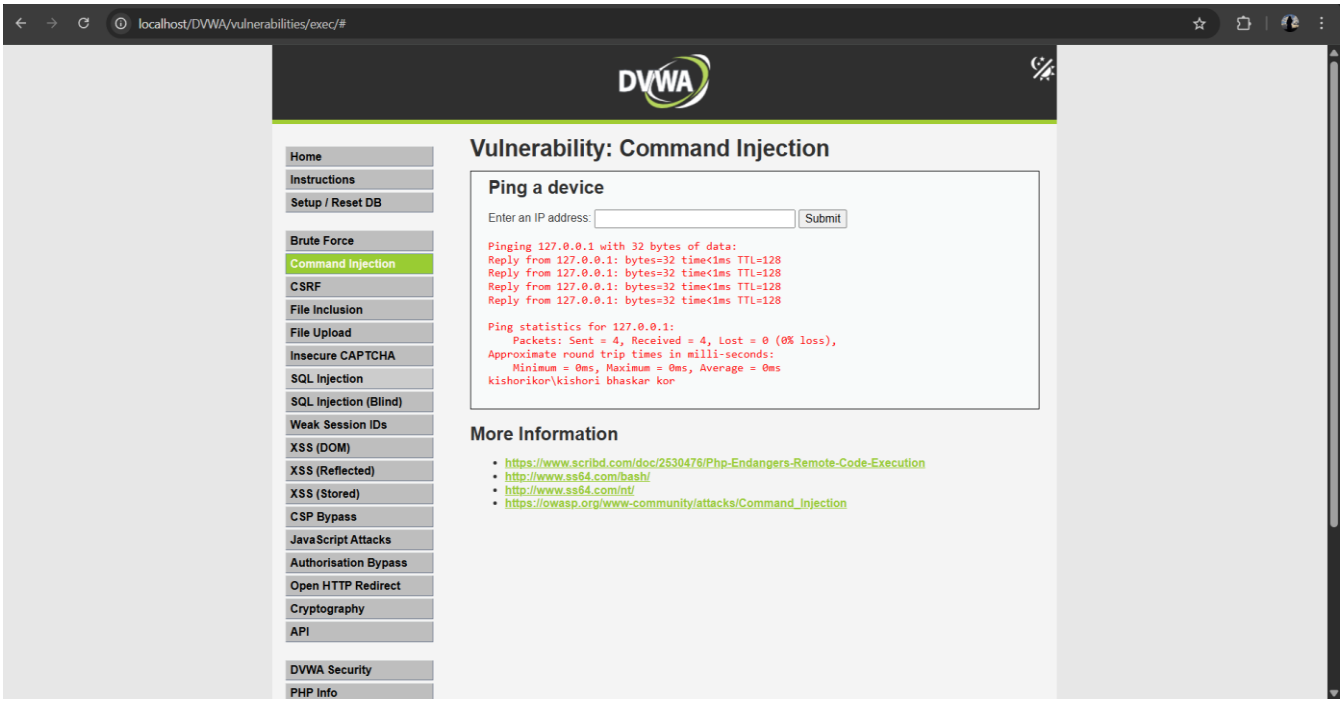
Description:
Command Injection is a vulnerability that allows attackers to execute system commands on the server through a web application.

Steps:

- Navigated to the **Command Injection** page in DVWA.
- In the IP address input field, entered the payload:

  127.0.0.1 & whoami
- Clicked **Submit**.

Result:
The application displayed the normal ping result and also showed the output of the whoami command. This confirms that the server executed an additional system command, proving the application is vulnerable to command injection.



Risk Level: Critical

Mitigation:

- Avoid executing system commands from user input
- Use secure APIs instead of shell commands
- Validate and sanitize input data
- Apply strict input filtering
- Run applications with minimal privileges

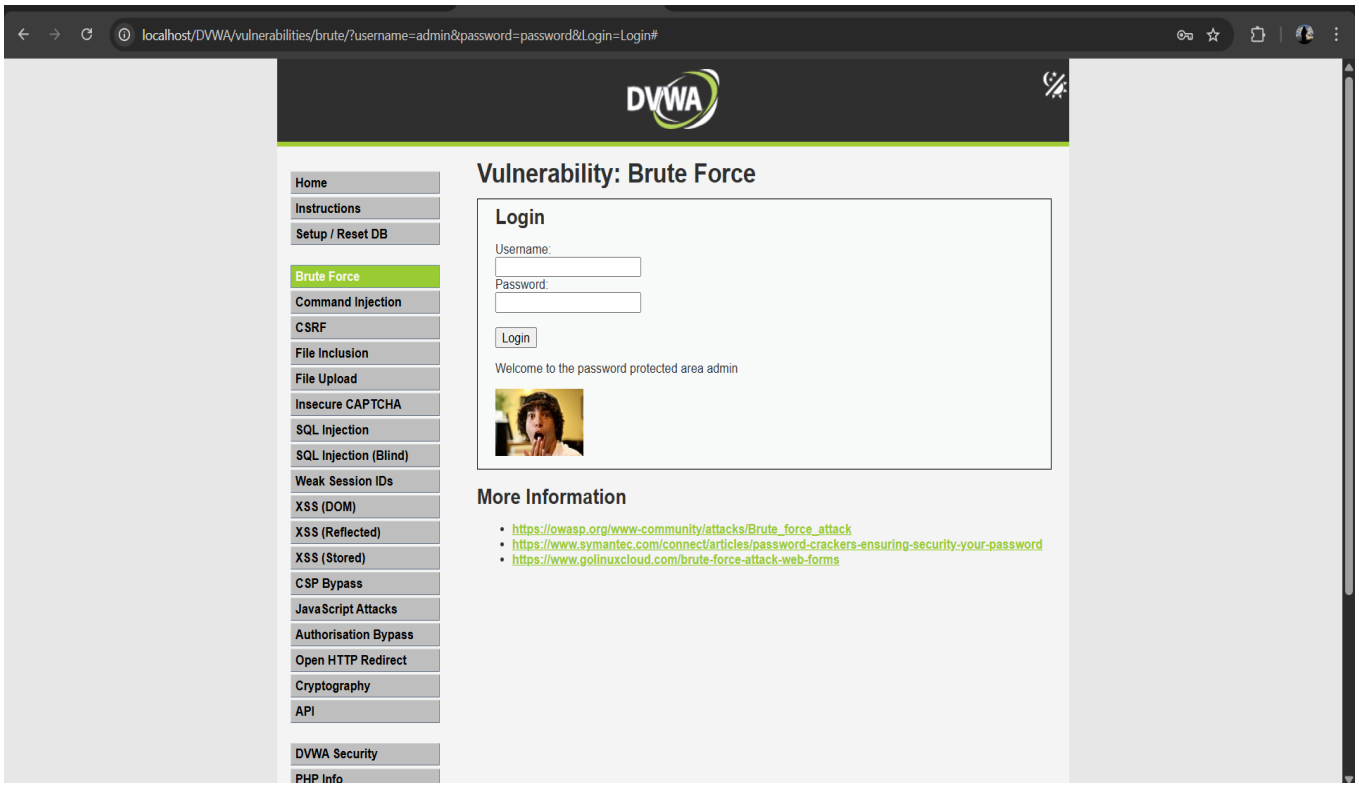
5. Brute Force Attack (Login Vulnerability)

Description: Automated attempts to guess passwords.

Steps:

- Navigated to DVWA Brute Force page.
- Attempted multiple login attempts using a list of passwords.

Result: Found valid credentials, showing weak password protection.



Risk Level: Medium

Mitigation:

- Implement account lockout after multiple failed attempts
- Use CAPTCHA verification
- Enforce strong password policies
- Implement multi-factor authentication (MFA)
- Rate-limit login attempts

CONCLUSION

This project demonstrated how common web application vulnerabilities such as SQL Injection, Cross-Site Scripting, File Upload, Command Injection, and Brute Force attacks can be exploited in a vulnerable environment. Performing VAPT helps security professionals identify weaknesses and improve application security before attackers can exploit them.